

Concurrency and Distributed Systems

Week 5 – Part 3: Thread Pools, Queues, and Rejection

Dr. Mohamad Aoude

Faculty of Engineering

May 11, 2026

Part 3 Roadmap

- 1 Thread Pool Policy
- 2 Fixed and Cached Pools
- 3 ThreadPoolExecutor
- 4 Saturation and Rejection
- 5 Sizing and Measurement
- 6 Lab Preparation

Why This Part Matters

Core question

Once work is queued, how many workers should execute it?

Lecture 05.02 bounded the waiting room. This lecture bounds the workers.

Week 5 principle

A thread pool is a resource budget, not just a faster way to run code.

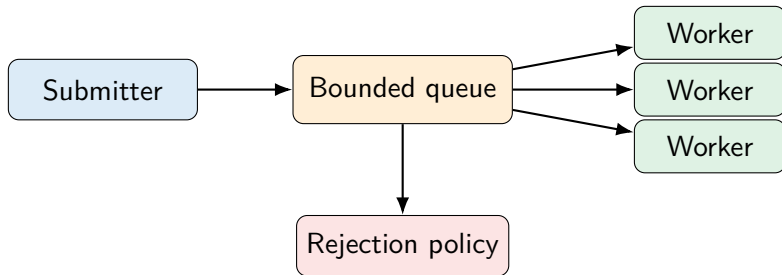
Execution Policy Revisited

- How many tasks run concurrently?
- How many tasks may wait?
- What happens when both workers and queue are full?
- How are threads created and named?
- How does shutdown complete?

ThreadPoolExecutor

Java exposes these choices directly through `ThreadPoolExecutor`.

Pool Anatomy



- Workers execute.
- The queue buffers.
- Rejection defines overload behavior.

Fixed Thread Pool

- Fixed number of worker threads.
- Predictable upper bound on concurrent execution.
- Good default for bounded service components.
- Needs deliberate queue capacity when built manually.

Use when

You know the resource budget and want stable behavior under load.

Fixed Pool Example

```
1 ExecutorService pool = Executors.newFixedThreadPool(2);  
2  
3 for (int i = 0; i < tasks; i++) {  
4     pool.submit(work);  
5 }
```

- Only two tasks run at once.
- Extra tasks wait.
- Worker reuse avoids per-task thread creation.

Cached Thread Pool

- Creates threads on demand.
- Reuses idle threads when available.
- Idle threads eventually time out.
- Can grow aggressively during bursts.

Risk

Cached pools are effectively unbounded unless another mechanism limits submissions.

Demo15: Cached Thread Pool

```
1 ExecutorService pool = Executors.newCachedThreadPool();
2 Set<String> workerNames = Collections.synchronizedSet(new HashSet<>());
3
4 for (int i = 0; i < tasks; i++) {
5     pool.submit(() -> {
6         workerNames.add(Thread.currentThread().getName());
7         sleep(50);
8         done.countDown();
9     });
10 }
```

What students should notice

Bursty submissions may use more worker threads than a fixed pool would.

Fixed vs Cached

Choice	Fixed pool	Cached pool
Worker bound	explicit	not explicit
Burst behavior	queues work	creates workers
Predictability	high	lower
Risk	queue delay	thread explosion
Best for	services with budgets	short bursty tasks

ThreadPoolExecutor Shape

```
1 ExecutorService pool = new ThreadPoolExecutor(  
2     corePoolSize,  
3     maximumPoolSize,  
4     keepAliveTime,  
5     TimeUnit.MILLISECONDS,  
6     workQueue,  
7     threadFactory,  
8     rejectionHandler);
```

- This is the configurable engine under most executor factories.
- Use it when factory methods hide a policy you need to control.

ThreadPoolExecutor Parameters

Parameter	Meaning
<code>corePoolSize</code>	Threads normally kept for work.
<code>maximumPoolSize</code>	Upper worker limit when the queue allows growth.
<code>keepAliveTime</code>	How long excess idle workers remain alive.
<code>workQueue</code>	Where waiting tasks are stored.
<code>ThreadFactory</code>	How worker threads are created.
<code>RejectedExecutionHandler</code>	What happens when saturated.

Demo05: BoundedExecutor

```
1 return new ThreadPoolExecutor(  
2     workers,  
3     workers,  
4     0L,  
5     TimeUnit.MILLISECONDS,  
6     new ArrayBlockingQueue<>(queueCapacity),  
7     new ThreadPoolExecutor.AbortPolicy());
```

- Fixed worker count.
- Bounded queue.
- Fail-fast rejection when full.

Saturation

Definition

A pool is saturated when it cannot immediately accept more useful work.

- All workers are busy.
- The work queue is full.
- A new task arrives.

Design question

What should the caller observe when this happens?

Demo06: Fail-Fast Rejection

```
1 ThreadPoolExecutor executor = Demo05_BoundedExecutor.create(1, 1);  
2  
3 executor.execute(() -> sleep(300)); // occupies worker  
4 executor.execute(() -> sleep(300)); // fills queue  
5 executor.execute(() -> { }); // rejected
```

- One worker is busy.
- One queue slot is filled.
- The third task triggers `RejectedExecutionException`.

Rejection Policies

Policy	Behavior
<code>AbortPolicy</code>	Throw <code>RejectedExecutionException</code> .
<code>CallerRunsPolicy</code>	Run task in the submitting thread.
<code>DiscardPolicy</code>	Drop the new task silently.
<code>DiscardOldestPolicy</code>	Drop oldest queued task, then retry.

Teaching point

Rejection is not an error in the design. It is the design's overload contract.

Choosing a Rejection Policy

- Use `AbortPolicy` when callers must know the request was not accepted.
- Use `CallerRunsPolicy` when slowing the caller is acceptable.
- Use discard policies only when losing work is part of the business rule.
- Always log or count rejections in service code.

Danger

Silent discard policies can turn overload into data loss.

Pool Sizing: CPU-Bound Work

- CPU-bound tasks spend most time computing.
- More runnable threads than cores creates scheduling overhead.
- Start near available processor count.
- Measure before increasing.

Rule of thumb

CPU-bound pool size $\approx$ number of cores, adjusted for measurement.

Pool Sizing: I/O-Bound Work

- I/O-bound tasks spend time waiting.
- More workers may hide waiting time.
- Too many workers still consume memory and scheduling overhead.
- Capacity must include downstream systems: database, disk, network.

Heuristic

Pool size often grows with wait time, but the only reliable answer is measurement under realistic load.

Sizing Formula Intuition

A common heuristic:

$$N_{threads} = N_{cpu} \times \left(1 + \frac{W}{C}\right)$$

- W : average wait time.
- C : average compute time.
- If tasks mostly wait, more threads may help.
- If tasks mostly compute, extra threads mostly contend.

Demo16: Fixed vs Cached Measurement

```
1 long fixed = run(Executors.newFixedThreadPool(fixedThreads),
2                 tasks, sleepMillis);
3 long cached = run(Executors.newCachedThreadPool(),
4                  tasks, sleepMillis);
5
6 return new Measurement(fixed, cached, tasks);
```

What students should notice

The "faster" policy depends on workload, burst size, and resource limits.

What to Measure

- Throughput: completed tasks per second.
- Latency: time per task or request.
- Queue depth: waiting work.
- Rejection count: overload events.
- Active worker count.
- Downstream pressure: database/socket/file waits.

Rule

Do not call a pool "good" from throughput alone.

Why Not Hand-Roll an Executor?

- Correct shutdown is subtle.
- Completion tracking is subtle.
- Cancellation is subtle.
- Exception propagation is subtle.
- Queue capacity and rejection semantics are easy to get wrong.

Course stance

Use standard executors for production. Hand-rolled executors are discussion material.

Demo17: Contrast Example

```
1 ToyFuture<String> first = executor.submit(task101);  
2 ToyFuture<String> second = executor.submit(task202);  
3  
4 String firstResult = first.get();  
5 String secondResult = second.get();  
6  
7 executor.shutdownAndAwait(1, TimeUnit.SECONDS);
```

- The toy executor shows how much machinery is needed.
- It is not the pattern students should copy.
- The lesson is to prefer `ThreadPoolExecutor`.

Exercise 2: BoundedExecutor

```
1 public final class Ex2_BoundedExecutor {  
2     public Ex2_BoundedExecutor(int workers, int queueCapacity) { ... }  
3     public Future<?> submit(Runnable task) { ... }  
4     public void shutdown() { ... }  
5     public boolean awaitTermination(long timeout, TimeUnit unit) { ... }  
6 }
```

- Build on `ThreadPoolExecutor`.
- Use bounded queue capacity.
- Expose a clear rejection behavior.
- Shut down cleanly.

Common Mistakes in Ex2

- Using `newCachedThreadPool()` where bounded behavior is required.
- Forgetting to shut down the pool in tests.
- Swallowing `RejectedExecutionException`.
- Using an unbounded `LinkedBlockingQueue`.
- Creating a new thread for each submitted task.

Lab Preparation

- Read Demo05_BoundedExecutor.
- Read Demo06_RejectionPolicyDemo.
- Run Demo15_CachedThreadPool.
- Run Demo16_FixedVsCachedThroughput.
- Treat Demo17 as instructor discussion, not student template code.

Part 3 Takeaway

Final takeaway

Thread pool design is capacity design: workers, queue, rejection, shutdown, and measurement must agree.

Bridge to Part 4

Once work is running under a bounded policy, the next question becomes:

How do we observe and optimize shared state?

Questions?